PulseAPI

API Monitoring and Alerting Platform

(Database Design & E-R Diagram)

1. Introduction:

- **PulseAPI** is an API Monitoring and Alerting Platform being developed to continuously monitor APIs and check their availability and performance.
- The system allows users to register APIs, configure monitoring intervals, store monitoring results, and generate alerts when an API has a problem.
- The database is designed to store user information, registered API details, monitoring results, and alert information. MySQL is used as the relational database management system (RDBMS) for the project.

2. Database Objectives:

- Store user registration and account information.
- Store APIs added by users for monitoring.
- Store repeated API monitoring results and response information.
- Store alert details generated for monitored APIs.
- Maintain relationships between users, APIs, monitoring results, and alerts.
- Support future dashboard, history, and reporting features.

3. Database Name:

- **Database Name:** pulseapi_db
- **DBMS:** MySQL

Main Entities:

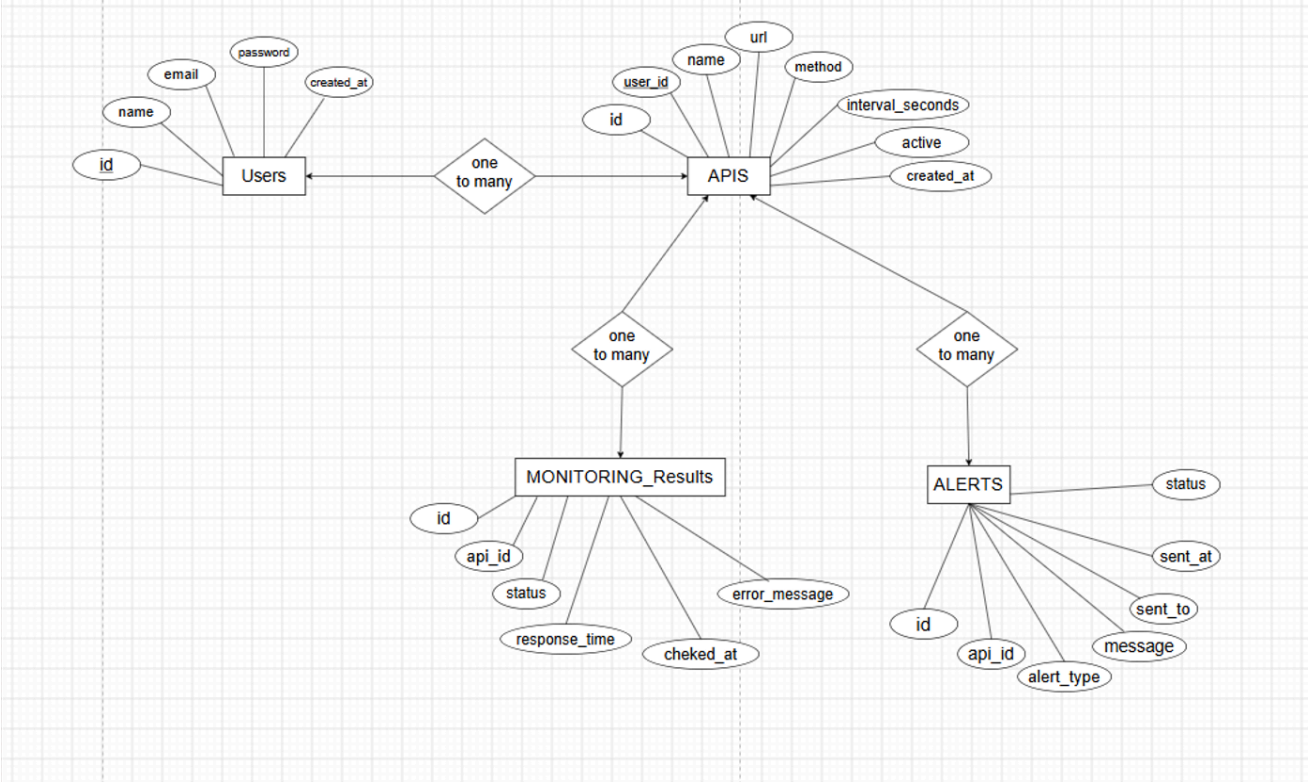
No.	Entity	Purpose
1.	Users	Stores registered user information
2.	APIs	Stores API details that users want to monitor
3.	Monitoring_Result	Stores the result of API health checks
4.	Alerts	Stores alerts generated for monitored APIs.

1. E-R Diagram (ERD)

- The E-R diagram represents the entities, attributes, relationships, and cardinality of the PulseAPI database.

  The four main entities are:

  ➔ Users
  ➔ APIs
  ➔ Monitoring_Results
  ➔ Alerts



❖ Entity and Attribute and Relationships

1. Users:

- **Purpose:** The users table stores the information of users who register and use the PulseAPI platform.
- **Fields :**

Fields	Key/Constraint	Description
user_id	Primary Key(PK)	Unique ID of the user
name	NOT NULL	Name of the user
email	UNIQUE, NOT NULL	Email address of the user.
password	NOT NULL	Password used for authentication.
created_at	Default Timestamp	Date and time when the user account was created.

- **Relationship with APIs:** One-to-Many (1:N)
- One **User** can have/register many **APIs**.
- Each **API** belongs to one **User**.
- users.id → apis.user_id

2. APIs :

- **Purpose:** The apis table stores information about the APIs added by users for monitoring.
- **Fields :**

Fields	Key/Constraint	Description
api_id	Primary Key (PK)	Unique ID of the API
user_id	Foreign Key (FK)	ID of the user who added the API
api_name	NOT NULL	Name of the API
api_url	NOT NULL	URL of the API
http_method	Default GET	HTTP method such as GET or POST
monitoring_interval	NOT NULL	Time interval for API monitoring
status	Default TRUE	Current API status
created_at	Default Timestamp	API creation date and time

- **Relationships with three entities/records:**
- **With Users:** Many-to-One (N:1) - Many APIs can belong to one User.
- **With Monitoring_Results:** One-to-Many (1:N) - One API can have many Monitoring Results because the API is checked repeatedly.
- **With Alerts:** One-to-Many (1:N) - One API can generate many Alerts when monitoring problems occur.

3. Monitoring_Result :

- **Purpose:**The monitoring_results table stores the results obtained when the system checks an API.
- It stores information such as response time, HTTP status code, API status, errors, and checking time.
- **Fields :**

Fields	Key/Constraint	Description
result_id	Primary Key (PK)	Unique monitoring result ID
api_id	Foreign Key (FK)	ID of the monitored API
status_code	NULL allowed	HTTP response code
response_time	NULL allowed	API response time
status	NULL allowed	UP or DOWN
error_message	NULL allowed	Error details if the API fails
checked_at	Default Timestamp	Time when the API was checked

- **Relationship with APIs: Many-to-One (N:1)**
  - One API can have many monitoring results.
  - Each monitoring result belongs to one API.
  - monitoring_results.api_id → apis.id

4. Alerts :

- **Purpose:**The **alerts** table stores information about alerts generated when an API fails or faces a monitoring problem.

- **Fields :**

Fields	Key/Constraint	Description
alert_id	Primary Key (PK)	ID of the API related to the alert
api_id	Foreign Key (FK)	ID of the monitored API
alert_type	NULL allowed	Type of alert
message	NULL allowed	Alert message
sent_at	Default Timestamp	Time when the alert was sent
alert_status	NULL allowed	Status of the alert

- **Relationship with APIs: Many-to-One (N:1)**
- One API can generate many alerts.
- Each alert belongs to one API.
- alerts.api_id → apis.id

5. Relationship and Cardinality:

The following relationships are present in the PulseAPI database.

Relationship	Cardinality	Explanation
Users → APIs	1:N	One user can add many APIs.
APIs → Monitoring_Results	1:N	One API can have many monitoring results because it is checked repeatedly.
APIs → ALERTS	1:N	One API can generate many alerts over time.

---

2. MySQL Table Creation

The following SQL structure matches the entities and relationships shown in the ER diagram.

1. Creating a Database:

CREATE DATABASE pulseapi_db;

```
mysql> CREATE DATABASE pulseapi_db;
Query OK, 1 row affected (0.02 sec)
```

2. Using a Database:

USE pulseapi_db;

```
mysql> USE pulseapi_db;
Database changed
```

3. Database Tables:

The PulseAPI database contains four main tables:

1. users
2. apis
3. monitoring_results
4. alerts

1. Users Table:

- **Description:**The users table stores the registration and account information of users.
- **SQL Query :**

```
mysql> CREATE TABLE users (
    -> user_id BIGINT PRIMARY KEY AUTO_INCREMENT,
    -> name VARCHAR(100) NOT NULL,
    -> email VARCHAR(150) UNIQUE NOT NULL,
    -> password VARCHAR(255) NOT NULL,
    -> created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    -> );
Query OK, 0 rows affected (0.07 sec)

mysql>
```

- **Screenshot 2: Users Table :**

```
mysql> DESC users;
+------------+--------------+------+-----+-------------------+-------------------+
| Field      | Type         | Null | Key | Default           | Extra             |
+------------+--------------+------+-----+-------------------+-------------------+
| user_id    | bigint       | NO   | PRI | NULL              | auto_increment    |
| name       | varchar(100) | NO   |     | NULL              |                   |
| email      | varchar(150) | NO   | UNI | NULL              |                   |
| password   | varchar(255) | NO   |     | NULL              |                   |
| created_at | timestamp    | YES  |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+------------+--------------+------+-----+-------------------+-------------------+
5 rows in set (0.01 sec)

mysql>
```

2. APIs Table:

- **Description:**

- The apis table stores the API details added by users.
- The user_id is a foreign key that connects the API with the user who added it.

- **SQL Query:**

```
mysql> CREATE TABLE apis (
    -> api_id BIGINT PRIMARY KEY AUTO_INCREMENT,
    -> user_id BIGINT NOT NULL,
    -> api_name VARCHAR(100) NOT NULL,
    -> api_url VARCHAR(500) NOT NULL,
    -> http_method VARCHAR(10) DEFAULT 'GET',
    -> monitoring_interval INT NOT NULL,
    -> status VARCHAR (20) DEFAULT 'UNKOWN',
    -> created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    -> FOREIGN KEY (user_id) REFERENCES users(user_id)
    -> );
Query OK, 0 rows affected (0.08 sec)
```

- **Screenshot 3: APIs Table:**

```
mysql> DESC apis;
+--------------------+--------------+------+-----+-------------------+-------------------+
| Field              | Type         | Null | Key | Default           | Extra             |
+--------------------+--------------+------+-----+-------------------+-------------------+
| api_id             | bigint       | NO   | PRI | NULL              | auto_increment    |
| user_id            | bigint       | NO   | MUL | NULL              |                   |
| api_name           | varchar(100) | NO   |     | NULL              |                   |
| api_url            | varchar(500) | NO   |     | NULL              |                   |
| http_method        | varchar(10)  | YES  |     | GET               |                   |
| monitoring_interval | int          | NO   |     | NULL              |                   |
| status             | varchar(20)  | YES  |     | UNKOWN            |                   |
| created_at         | timestamp    | YES  |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+--------------------+--------------+------+-----+-------------------+-------------------+
8 rows in set (0.00 sec)

mysql>
```

3. Monitoring Results Table :

- **Description:**

- The monitoring_results table stores the result of each API health check.
- It stores the API status, response time, checking time, and error message if the API fails.

- **SQL Query:**

```
mysql> CREATE TABLE monitoring_results (
    -> result_id BIGINT PRIMARY KEY AUTO_INCREMENT,
    -> api_id BIGINT NOT NULL,
    -> status_code INT,
    -> response_time BIGINT,
    -> status VARCHAR(20),
    -> error_message VARCHAR(500),
    -> checked_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    -> FOREIGN KEY (api_id) REFERENCES apis(api_id)
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql>
```

Screenshot 4: Monitoring Results Table:

```
mysql> DESC monitoring_results;
+---------------+--------------+------+-----+-------------------+-------------------+
| Field         | Type         | Null | Key | Default           | Extra             |
+---------------+--------------+------+-----+-------------------+-------------------+
| result_id     | bigint       | NO   | PRI | NULL              | auto_increment    |
| api_id        | bigint       | NO   | MUL | NULL              |                   |
| status_code   | int          | YES  |     | NULL              |                   |
| response_time | bigint       | YES  |     | NULL              |                   |
| status        | varchar(20)  | YES  |     | NULL              |                   |
| error_message | varchar(500) | YES  |     | NULL              |                   |
| checked_at    | timestamp    | YES  |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+---------------+--------------+------+-----+-------------------+-------------------+
7 rows in set (0.00 sec)

mysql>
```

4. Alerts Table:

- **Description:**

- The alerts table stores information about alerts generated when an API fails or faces a monitoring problem.
- The api_id is a foreign key that connects the alert with the related API.

- **SQL Query:**

```
mysql> CREATE TABLE alerts (
    -> alert_id BIGINT PRIMARY KEY AUTO_INCREMENT,
    -> api_id BIGINT NOT NULL,
    -> alert_type VARCHAR(50),
    -> message VARCHAR(500),
    -> sent_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    -> alert_status VARCHAR(20),
    -> FOREIGN KEY (api_id) REFERENCES apis(api_id)
    -> );
Query OK, 0 rows affected (0.07 sec)

mysql>
```

- **Screenshot 5: Alerts Table:**

```
mysql> DESC alerts;
+--------------+--------------+------+-----+-------------------+-------------------+
| Field        | Type         | Null | Key | Default           | Extra             |
+--------------+--------------+------+-----+-------------------+-------------------+
| alert_id     | bigint       | NO   | PRI | NULL              | auto_increment    |
| api_id       | bigint       | NO   | MUL | NULL              |                   |
| alert_type   | varchar(50)  | YES  |     | NULL              |                   |
| message      | varchar(500) | YES  |     | NULL              |                   |
| sent_at      | timestamp    | YES  |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
| alert_status | varchar(20)  | YES  |     | NULL              |                   |
+--------------+--------------+------+-----+-------------------+-------------------+
6 rows in set (0.00 sec)

mysql>
```

5. Database Verification:

After creating all the tables, the following command is used to verify the tables:

SHOW TABLES;

The database should contain:

- Users
- Apis
- Monitoring_results
- alerts

- **Screenshot 6: Database Tables**